

Odor & Weather Variable Correlation Analysis

This notebook performs an advanced non-linear correlation and regression analysis between odor complaints and meteorological variables for both **Pittsburgh, PA** and **Louisville, KY** datasets.

It focuses on:

1. **Baseline-frequency-adjusted rates** to eliminate bias due to weather frequency distributions.
2. **Multivariate Poisson regression marginal effects overlays** to show the true predictive curves of the models.
3. **Advanced non-linear correlation metrics** (Spearman Rank, Mutual Information, Wald z-scores, and Drop-in-Deviance) to highlight non-linear meteorological patterns.

```
In [1]: import os
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr, spearmanr
from sklearn.feature_selection import mutual_info_regression
import warnings

warnings.filterwarnings('ignore')
sns.set_theme(style="whitegrid")
print("Libraries successfully imported!")
```

Libraries successfully imported!

Part 1: Pittsburgh, PA Analysis

```
In [2]: # Load Pittsburgh merged hourly weather and complaints data
pitt_file = 'Pittsburgh Data/open-meteo-smell-merged.csv'
print(f"Reading dataset from: {pitt_file}")
df_raw_pitt = pd.read_csv(pitt_file)

# Column mapping for Pittsburgh
column_mapping = {
    'time': 'datetime',
    'smell_report_count': 'complaints',
    'temperature_2m (°F)': 'temperature',
    'relative_humidity_2m (%)': 'relative_humidity',
    'wind_speed_10m (mp/h)': 'wind_speed',
```

```

        'wind_direction_10m (°)': 'wind_direction',
        'rain (inch)': 'precipitation',
        'dew_point_2m (°F)': 'dew_point',
        'vapour_pressure_deficit (kPa)': 'vapor_pressure',
        'surface_pressure (hPa)': 'atmospheric_pressure',
        'shortwave_radiation (W/m²)': 'solar_radiation',
        'sunshine_duration (s)': 'sunshine_duration',
        'boundary_layer_height (ft)': 'boundary_layer_height'
    }
df_mapped = df_raw_pitt.rename(columns=column_mapping)
df_mapped['datetime'] = pd.to_datetime(df_mapped['datetime'])

# Aggregate hourly
df_pitt = df_mapped.groupby('datetime').agg({
    'complaints': 'sum',
    'temperature': 'mean',
    'precipitation': 'mean',
    'wind_speed': 'mean',
    'wind_direction': 'mean',
    'dew_point': 'mean',
    'relative_humidity': 'mean',
    'vapor_pressure': 'mean',
    'atmospheric_pressure': 'mean',
    'sunshine_duration': 'mean',
    'solar_radiation': 'mean',
    'boundary_layer_height': 'mean'
}).reset_index()

df_pitt['date'] = df_pitt['datetime'].dt.date
df_pitt['dayofweek'] = df_pitt['datetime'].dt.dayofweek
df_pitt['is_weekend'] = df_pitt['dayofweek'].isin([5, 6])

# Aggregate daily (using all daily complaints)
daily_df_pitt = df_pitt.groupby('date').agg({
    'complaints': 'sum',
    'is_weekend': 'first',
    'temperature': ['mean', 'min', 'max'],
    'precipitation': 'sum',
    'wind_speed': 'mean',
    'relative_humidity': 'mean',
    'atmospheric_pressure': 'mean',
    'solar_radiation': 'mean',
    'boundary_layer_height': 'mean'
}).reset_index()

daily_df_pitt.columns = [
    'date', 'complaints', 'is_weekend',
    'temperature', 'temp_min', 'temp_max',
    'precipitation', 'wind_speed',
    'relative_humidity', 'atmospheric_pressure', 'solar_radiation', 'boundary_layer_height'
]

daily_df_pitt['diurnal_temperature_range'] = daily_df_pitt['temp_max'] - daily_df_pitt['temp_min']
daily_df_pitt['temperature_squared'] = daily_df_pitt['temperature'] ** 2
daily_df_pitt['is_weekend'] = daily_df_pitt['is_weekend'].astype(int)

```

```

model_vars = ['temperature', 'temperature_squared', 'solar_radiation', 'rela
              'wind_speed', 'precipitation', 'diurnal_temperature_range',
              'boundary_layer_height', 'atmospheric_pressure', 'is_weekend']

df_count_pitt = daily_df_pitt[['complaints'] + model_vars].dropna()
print("Pittsburgh Daily Prep Completed! Shape:", df_count_pitt.shape)

```

Reading dataset from: Pittsburgh Data/open-meteo-smell-merged.csv
Pittsburgh Daily Prep Completed! Shape: (2899, 11)

```

In [3]: X_count_pitt = df_count_pitt[model_vars]
X_count_pitt = sm.add_constant(X_count_pitt)
y_count_pitt = df_count_pitt['complaints']

poisson_model_pitt = sm.GLM(y_count_pitt, X_count_pitt, family=sm.families.F
poisson_res_pitt = poisson_model_pitt.fit()
print("Pittsburgh Poisson Regression Results Summary:")
print(poisson_res_pitt.summary())

```

Pittsburgh Poisson Regression Results Summary:
Generalized Linear Model Regression Results

```
=====
==
Dep. Variable:          complaints    No. Observations:          28
99
Model:                  GLM          Df Residuals:              28
88
Model Family:          Poisson      Df Model:
10
Link Function:          Log          Scale:                  1.00
00
Method:                 IRLS         Log-Likelihood:         -3194
7.
Date:                   Tue, 09 Jun 2026    Deviance:              4967
0.
Time:                   18:59:41          Pearson chi2:          6.17e+
04
No. Iterations:         5              Pseudo R-squ. (CS):    1.0
00
Covariance Type:        nonrobust
=====
```

```
=====
                                coef      std err          z      P>|z|
[0.025      0.975]
-----
const                        12.9171      0.689      18.747      0.000      1
1.567      14.268
temperature                   0.0588      0.001      45.424      0.000
0.056      0.061
temperature_squared          -0.0004      1.26e-05     -31.729      0.000      -
0.000      -0.000
solar_radiation             -0.0037      7.73e-05     -47.479      0.000      -
0.004      -0.004
relative_humidity            -0.0161      0.000     -38.209      0.000      -
0.017      -0.015
wind_speed                   -0.0252      0.002     -10.397      0.000      -
0.030      -0.020
precipitation                -0.4951      0.019     -25.940      0.000      -
0.533      -0.458
diurnal_temperature_range     0.0696      0.001     107.600      0.000
0.068      0.071
boundary_layer_height        -0.0001      2.25e-06     -64.524      0.000      -
0.000      -0.000
atmospheric_pressure         -0.0099      0.001     -14.522      0.000      -
0.011      -0.009
is_weekend                   -0.1465      0.007     -20.290      0.000      -
0.161      -0.132
=====
```

```
In [4]: target_weather_vars = [
        'temperature',
        'solar_radiation',
        'relative_humidity',
```

```

        'wind_speed',
        'precipitation',
        'diurnal_temperature_range',
        'boundary_layer_height',
        'atmospheric_pressure'
    ]

def bin_variable(series, name):
    if name == 'temperature':
        return series.round()
    elif name == 'precipitation':
        return (series / 0.05).round() * 0.05
    elif name == 'wind_speed':
        return series.round()
    elif name == 'relative_humidity':
        return series.round()
    elif name == 'atmospheric_pressure':
        return series.round()
    elif name == 'solar_radiation':
        return (series / 20).round() * 20
    elif name == 'boundary_layer_height':
        return (series / 500).round() * 500
    elif name == 'diurnal_temperature_range':
        return series.round()
    return series

# Pre-calculate binned columns
for var in target_weather_vars:
    df_count_pitt[f'{var}_binned'] = bin_variable(df_count_pitt[var], var)

n_vars = len(target_weather_vars)
n_cols = 3
n_rows = (n_vars + n_cols - 1) // n_cols

# --- SECTION 1: Normalized Actuals Only ---
fig1, axes1 = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes1 = axes1.flatten()

for i, var in enumerate(target_weather_vars):
    ax = axes1[i]
    binned_col = f'{var}_binned'
    binned_data = df_count_pitt.groupby([binned_col, 'is_weekend']).agg(
        avg_complaints=('complaints', 'mean'),
        num_days=('complaints', 'count')
    ).reset_index()
    binned_data = binned_data[binned_data['num_days'] >= 1]

    # Split weekdays and weekends
    wd_data = binned_data[binned_data['is_weekend'] == 0]
    we_data = binned_data[binned_data['is_weekend'] == 1]

    ax.scatter(wd_data[binned_col], wd_data['avg_complaints'],
               color='#8b2500', alpha=0.85, edgecolor='k', s=50, label='Week')
    ax.scatter(we_data[binned_col], we_data['avg_complaints'],
               color='#1f77b4', alpha=0.85, edgecolor='k', s=50, label='Week')

```

```

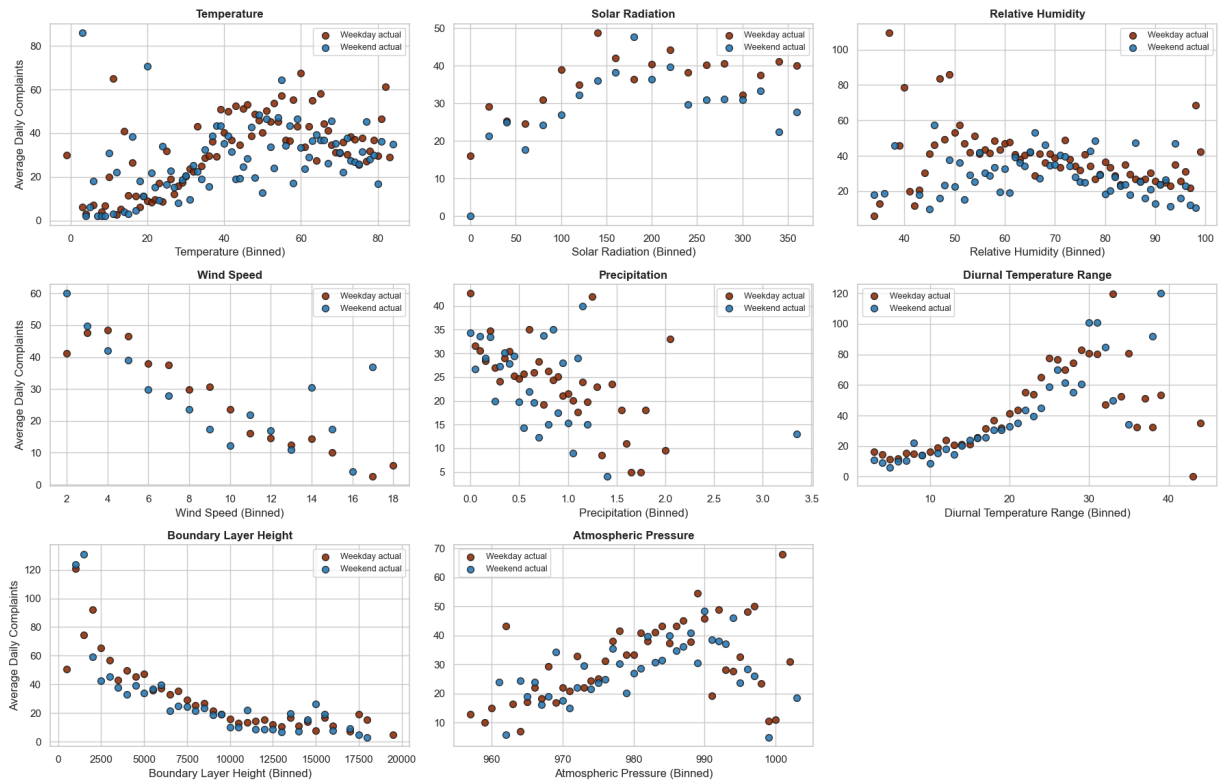
ax.set_title(var.replace('_', ' ').title(), fontsize=12, fontweight='bold')
ax.set_xlabel(var.replace('_', ' ').title() + ' (Binned)')
ax.set_ylabel('Average Daily Complaints' if i % n_cols == 0 else '')
ax.legend(fontsize=9, loc='best')

for j in range(i + 1, len(axes1)):
    axes1[j].set_visible(False)

fig1.suptitle('Pittsburgh Section 1: Normalized Actual Odor Complaints vs Weather Variables',
              fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

Pittsburgh Section 1: Normalized Actual Odor Complaints vs Weather Variables



```

In [5]: # --- SECTION 2: Multivariate Model Marginal Effects Overlay ---
medians_pitt = df_count_pitt[model_vars].median()

fig2, axes2 = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes2 = axes2.flatten()

for i, var in enumerate(target_weather_vars):
    ax = axes2[i]
    binned_col = f'{var}_binned'
    binned_data = df_count_pitt.groupby([binned_col, 'is_weekend']).agg(
        avg_complaints=('complaints', 'mean'),
        num_days=('complaints', 'count')
    ).reset_index()
    binned_data = binned_data[binned_data['num_days'] >= 1]

    # Split weekdays and weekends for background
    wd_data = binned_data[binned_data['is_weekend'] == 0]
    we_data = binned_data[binned_data['is_weekend'] == 1]

```

```

# Background scatter
ax.scatter(wd_data[binned_col], wd_data['avg_complaints'],
           color='#8b2500', alpha=0.4, edgecolor='k', s=50, label='Weekdays')
ax.scatter(we_data[binned_col], we_data['avg_complaints'],
           color='#1f77b4', alpha=0.4, edgecolor='k', s=50, label='Weekends')

# Generate smooth sequence
var_min = df_count_pitt[var].min()
var_max = df_count_pitt[var].max()
var_seq = np.linspace(var_min, var_max, 200)

# Weekday Prediction Line
pred_df_wd = pd.DataFrame(index=range(200))
pred_df_wd['const'] = 1.0
for m_var in model_vars:
    if m_var == var:
        pred_df_wd[m_var] = var_seq
    elif var == 'temperature' and m_var == 'temperature_squared':
        pred_df_wd[m_var] = var_seq ** 2
    elif m_var == 'is_weekend':
        pred_df_wd[m_var] = 0
    else:
        pred_df_wd[m_var] = medians_pitt[m_var]

pred_X_wd = pred_df_wd[['const'] + model_vars]
pred_y_wd = poisson_res_pitt.predict(pred_X_wd)

# Weekend Prediction Line
pred_df_we = pd.DataFrame(index=range(200))
pred_df_we['const'] = 1.0
for m_var in model_vars:
    if m_var == var:
        pred_df_we[m_var] = var_seq
    elif var == 'temperature' and m_var == 'temperature_squared':
        pred_df_we[m_var] = var_seq ** 2
    elif m_var == 'is_weekend':
        pred_df_we[m_var] = 1
    else:
        pred_df_we[m_var] = medians_pitt[m_var]

pred_X_we = pred_df_we[['const'] + model_vars]
pred_y_we = poisson_res_pitt.predict(pred_X_we)

# Overlay continuous prediction lines
ax.plot(var_seq, pred_y_wd, color='#8b2500', linewidth=2.5, label='Weekdays')
ax.plot(var_seq, pred_y_we, color='#1f77b4', linewidth=2.5, label='Weekends')

# Align Y-axis scales
max_actual_wd = wd_data['avg_complaints'].max() if not wd_data.empty else 0
max_actual_we = we_data['avg_complaints'].max() if not we_data.empty else 0
max_y = max(max_actual_wd, max_actual_we, pred_y_wd.max(), pred_y_we.max())
ax.set_ylim(-0.02 * max_y, max_y * 1.08)

ax.set_title(var.replace('_', ' ').title(), fontsize=12, fontweight='bold')
ax.set_xlabel(var.replace('_', ' ').title() + ' (Continuous / Binned)')

```

```

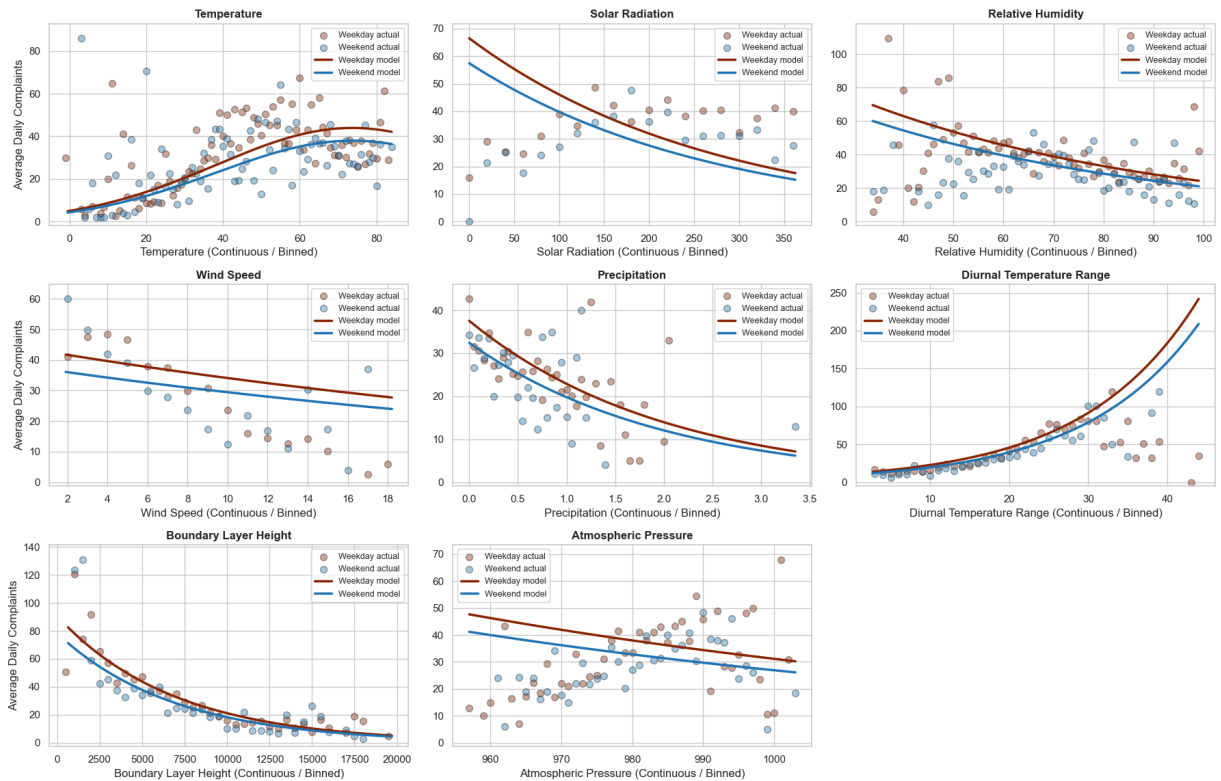
ax.set_ylabel('Average Daily Complaints' if i % n_cols == 0 else '')
ax.legend(fontsize=9, loc='best')

for j in range(i + 1, len(axes2)):
    axes2[j].set_visible(False)

fig2.suptitle('Pittsburgh Section 2: Multivariate Poisson Model Marginal Eff
              fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

Pittsburgh Section 2: Multivariate Poisson Model Marginal Effects Overlay



```

In [6]: # --- SECTION 2: Advanced Non-Linear Correlation Metrics ---
print("Calculating advanced correlation metrics for Pittsburgh...")

# 1. Pearson and Spearman correlations
pearson_pitt = {}
spearman_pitt = {}
for var in target_weather_vars:
    r_val, _ = pearsonr(df_count_pitt['complaints'], df_count_pitt[var])
    rho_val, _ = spearmanr(df_count_pitt['complaints'], df_count_pitt[var])
    pearson_pitt[var] = r_val
    spearman_pitt[var] = rho_val

# 2. Mutual Information (MI)
X_mi_pitt = df_count_pitt[target_weather_vars]
y_mi_pitt = df_count_pitt['complaints']
mi_scores_pitt = mutual_info_regression(X_mi_pitt, y_mi_pitt, random_state=4)
mi_pitt = dict(zip(target_weather_vars, mi_scores_pitt))

# 3. Model z-scores and Drop-in-Deviance
tvalues_pitt = poisson_res_pitt.tvalues

```



```

z_pitt = {}
dev_pitt = {}
full_dev_pitt = poisson_res_pitt.deviance

for var in target_weather_vars:
    if var == 'temperature':
        z_pitt[var] = max(abs(tvalues_pitt['temperature']), abs(tvalues_pitt[
            vars_keep = [v for v in model_vars if v != 'temperature' and v != 't
    else:
        z_pitt[var] = abs(tvalues_pitt[var])
        vars_keep = [v for v in model_vars if v != var]

    X_red = sm.add_constant(df_count_pitt[vars_keep])
    red_model = sm.GLM(y_count_pitt, X_red, family=sm.families.Poisson())
    red_res = red_model.fit()
    dev_pitt[var] = red_res.deviance - full_dev_pitt

print("Pittsburgh metrics successfully computed!")

```

Calculating advanced correlation metrics for Pittsburgh...
Pittsburgh metrics successfully computed!

```

In [7]: # --- SECTION 3: Correlation Metrics Visualization ---
pitt_comp = pd.DataFrame({
    'Variable': target_weather_vars,
    'Pearson r': [pearson_pitt[v] for v in target_weather_vars],
    'Spearman rho': [spearman_pitt[v] for v in target_weather_vars],
    'Mutual Information': [mi_pitt[v] for v in target_weather_vars],
    'Wald z-score': [z_pitt[v] for v in target_weather_vars],
    'Drop-in-Deviance': [dev_pitt[v] for v in target_weather_vars]
})

# Horizontal bar chart of Mutual Information (MI)
mi_sorted = pitt_comp.sort_values(by='Mutual Information', ascending=False)
plt.figure(figsize=(10, 5))
sns.barplot(x='Mutual Information', y='Variable', data=mi_sorted, palette='v
plt.title('Pittsburgh: Mutual Information (MI) Score by Weather Variable', f
plt.xlabel('Mutual Information Score (Non-Linear Impact)')
plt.ylabel('Meteorological Variable')
plt.tight_layout()
plt.show()

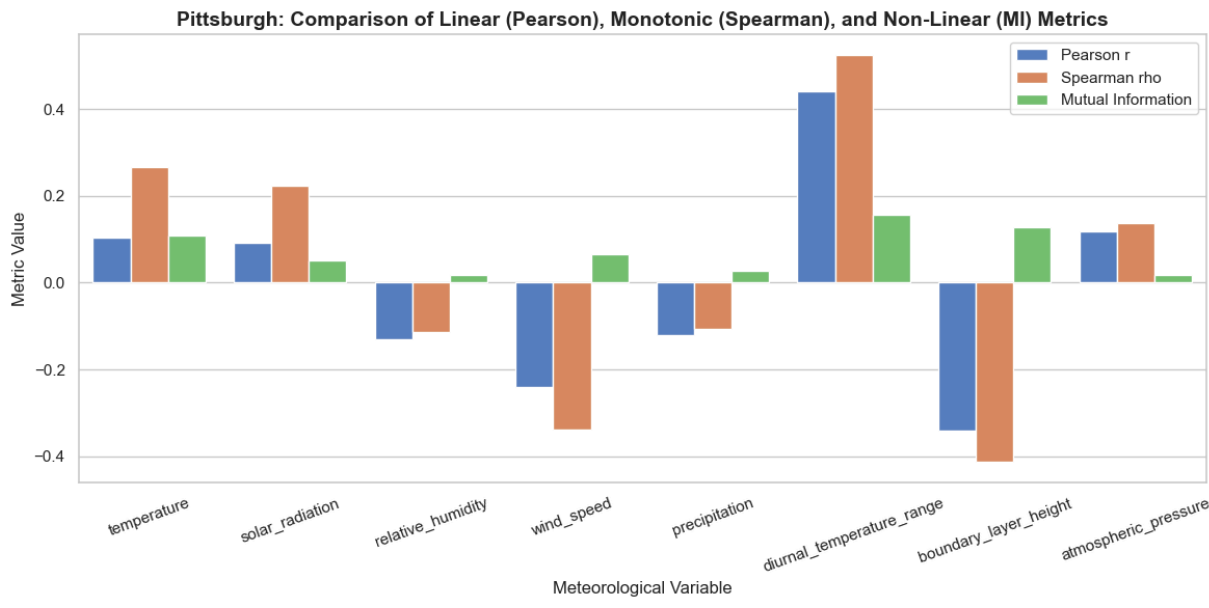
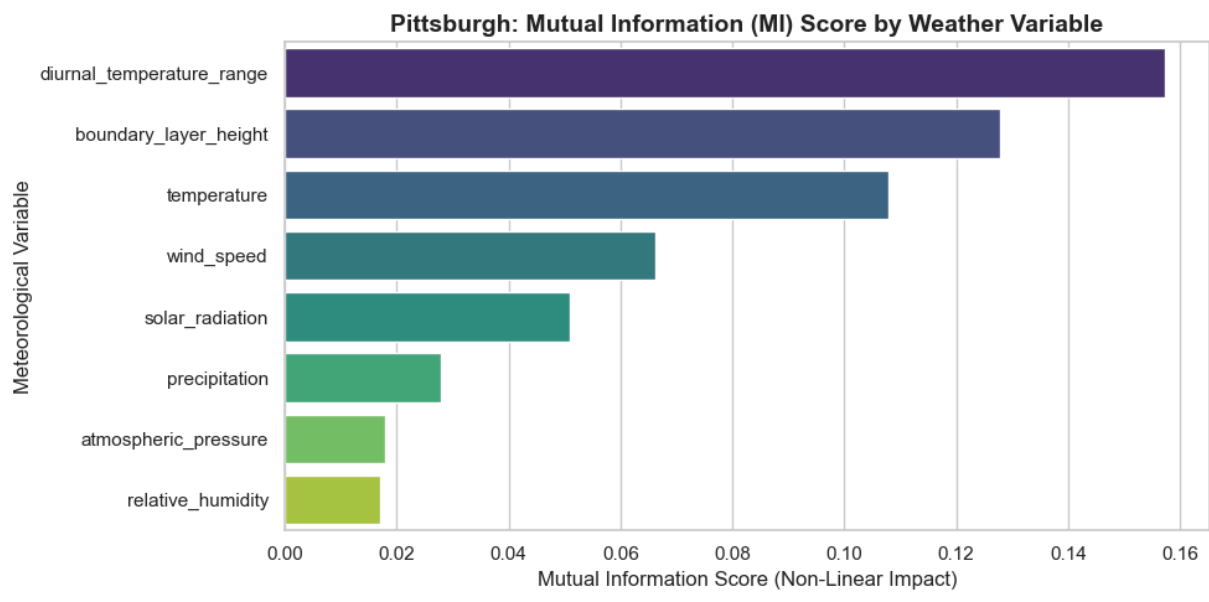
# Melt for comparison of Pearson, Spearman, and MI
melted_pitt = pitt_comp.melt(id_vars='Variable', value_vars=['Pearson r', 'S
    var_name='Metric', value_name='Value')

plt.figure(figsize=(12, 6))
sns.barplot(x='Variable', y='Value', hue='Metric', data=melted_pitt, palette
plt.title('Pittsburgh: Comparison of Linear (Pearson), Monotonic (Spearman),
plt.xlabel('Meteorological Variable')
plt.ylabel('Metric Value')
plt.xticks(rotation=20)
plt.legend(loc='best')
plt.tight_layout()
plt.show()

print("Pittsburgh Correlation Metrics Summary Table:")

```

```
pitt_comp.style.background_gradient(cmap='coolwarm', subset=['Pearson r', 'Spearman rho', 'Mutual Information'])
pitt_comp.style.background_gradient(cmap='viridis', subset=['Mutual Information'])
pitt_comp.format({
    'Pearson r': '{:.3f}',
    'Spearman rho': '{:.3f}',
    'Mutual Information': '{:.3f}',
    'Wald z-score': '{:.2f}',
    'Drop-in-Deviance': '{:.1f}'
})
```



Pittsburgh Correlation Metrics Summary Table:

Out [7]:

	Variable	Pearson r	Spearman rho	Mutual Information	Wald z- score	Drop-in- Deviance
0	temperature	0.104	0.267	0.108	45.42	5389.1
1	solar_radiation	0.092	0.223	0.051	47.48	2280.5
2	relative_humidity	-0.130	-0.114	0.017	38.21	1431.8
3	wind_speed	-0.241	-0.340	0.066	10.40	108.1
4	precipitation	-0.120	-0.107	0.028	25.94	728.4
5	diurnal_temperature_range	0.441	0.525	0.157	107.60	10627.6
6	boundary_layer_height	-0.340	-0.413	0.128	64.52	4366.2
7	atmospheric_pressure	0.119	0.138	0.018	14.52	210.5

Part 2: Louisville, KY Analysis

```
In [8]: # Load Louisville merged hourly weather and complaints data
lville_file = 'Louisville Data/open-meteo-smell-merged.csv'
print(f"Reading dataset from: {lville_file}")
df_raw_lville = pd.read_csv(lville_file)

# Column mapping for Louisville
df_mapped_l = df_raw_lville.rename(columns=column_mapping)
df_mapped_l['datetime'] = pd.to_datetime(df_mapped_l['datetime'])

# Aggregate hourly
df_lville = df_mapped_l.groupby('datetime').agg({
    'complaints': 'sum',
    'temperature': 'mean',
    'precipitation': 'mean',
    'wind_speed': 'mean',
    'wind_direction': 'mean',
    'dew_point': 'mean',
    'relative_humidity': 'mean',
    'vapor_pressure': 'mean',
    'atmospheric_pressure': 'mean',
    'sunshine_duration': 'mean',
    'solar_radiation': 'mean',
    'boundary_layer_height': 'mean'
}).reset_index()

df_lville['date'] = df_lville['datetime'].dt.date
df_lville['dayofweek'] = df_lville['datetime'].dt.dayofweek
df_lville['is_weekend'] = df_lville['dayofweek'].isin([5, 6])

# Aggregate daily (using all daily complaints)
daily_df_lville = df_lville.groupby('date').agg({
    'complaints': 'sum',
    'is_weekend': 'first',
    'temperature': ['mean', 'min', 'max'],
```

```

        'precipitation': 'sum',
        'wind_speed': 'mean',
        'relative_humidity': 'mean',
        'atmospheric_pressure': 'mean',
        'solar_radiation': 'mean',
        'boundary_layer_height': 'mean'
    }).reset_index()

daily_df_lville.columns = [
    'date', 'complaints', 'is_weekend',
    'temperature', 'temp_min', 'temp_max',
    'precipitation', 'wind_speed',
    'relative_humidity', 'atmospheric_pressure', 'solar_radiation', 'boundary_layer_height'
]

daily_df_lville['diurnal_temperature_range'] = daily_df_lville['temp_max'] - daily_df_lville['temp_min']
daily_df_lville['temperature_squared'] = daily_df_lville['temperature'] ** 2
daily_df_lville['is_weekend'] = daily_df_lville['is_weekend'].astype(int)

df_count_lville = daily_df_lville[['complaints'] + model_vars].dropna()
print("Louisville Daily Prep Completed! Shape:", df_count_lville.shape)

```

Reading dataset from: Louisville Data/open-meteo-smell-merged.csv
Louisville Daily Prep Completed! Shape: (2899, 11)

In [9]:

```

X_count_lville = df_count_lville[model_vars]
X_count_lville = sm.add_constant(X_count_lville)
y_count_lville = df_count_lville['complaints']

poisson_model_lville = sm.GLM(y_count_lville, X_count_lville, family=sm.fam.poisson)
poisson_res_lville = poisson_model_lville.fit()
print("Louisville Poisson Regression Results Summary:")
print(poisson_res_lville.summary())

```

Louisville Poisson Regression Results Summary:
Generalized Linear Model Regression Results

```
=====
==
Dep. Variable:          complaints    No. Observations:          28
99
Model:                  GLM          Df Residuals:              28
88
Model Family:           Poisson      Df Model:
10
Link Function:          Log          Scale:                  1.00
00
Method:                 IRLS         Log-Likelihood:          -947
6.8
Date:                   Tue, 09 Jun 2026    Deviance:              1311
7.
Time:                   18:59:46          Pearson chi2:          2.09e+
04
No. Iterations:         6              Pseudo R-squ. (CS):    0.77
22
Covariance Type:        nonrobust
=====
```

```
=====
coef      std err          z      P>|z|
-----
[0.025      0.975]
-----
const      18.0562      2.578      7.003      0.000      1
3.003      23.110
temperature      0.1875      0.006      30.011      0.000
0.175      0.200
temperature_squared      -0.0014      5.37e-05      -25.480      0.000      -
0.001      -0.001
solar_radiation      -0.0077      0.000      -32.007      0.000      -
0.008      -0.007
relative_humidity      -0.0466      0.001      -34.091      0.000      -
0.049      -0.044
wind_speed      -0.1842      0.007      -26.466      0.000      -
0.198      -0.171
precipitation      -0.2888      0.055      -5.281      0.000      -
0.396      -0.182
diurnal_temperature_range      0.0253      0.002      10.857      0.000
0.021      0.030
boundary_layer_height      7.062e-05      6.82e-06      10.352      0.000      5.7
3e-05      8.4e-05
atmospheric_pressure      -0.0179      0.002      -7.175      0.000      -
0.023      -0.013
is_weekend      -0.4929      0.027      -18.578      0.000      -
0.545      -0.441
=====
=====
```

```
In [10]: # Pre-calculate binned columns for Louisville
for var in target_weather_vars:
    df_count_lvllle[f'{var}_binned'] = bin_variable(df_count_lvllle[var], va
```

```

# --- SECTION 1: Normalized Actuals Only ---
fig3, axes3 = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes3 = axes3.flatten()

for i, var in enumerate(target_weather_vars):
    ax = axes3[i]
    binned_col = f'{var}_binned'
    binned_data = df_count_lville.groupby([binned_col, 'is_weekend']).agg(
        avg_complaints=('complaints', 'mean'),
        num_days=('complaints', 'count')
    ).reset_index()
    binned_data = binned_data[binned_data['num_days'] >= 1]

    # Split weekdays and weekends
    wd_data = binned_data[binned_data['is_weekend'] == 0]
    we_data = binned_data[binned_data['is_weekend'] == 1]

    ax.scatter(wd_data[binned_col], wd_data['avg_complaints'],
               color='#8b2500', alpha=0.85, edgecolor='k', s=50, label='Week')
    ax.scatter(we_data[binned_col], we_data['avg_complaints'],
               color='#1f77b4', alpha=0.85, edgecolor='k', s=50, label='Week')

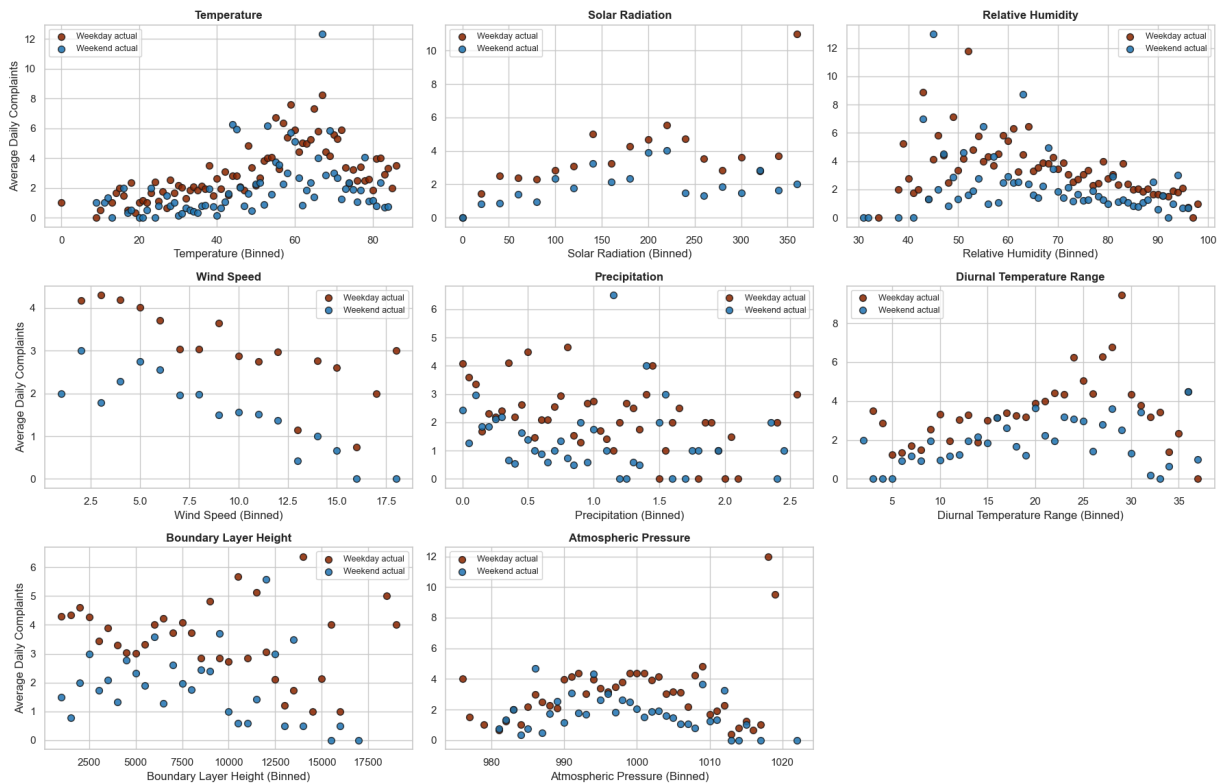
    ax.set_title(var.replace('_', ' ').title(), fontsize=12, fontweight='bold')
    ax.set_xlabel(var.replace('_', ' ').title() + ' (Binned)')
    ax.set_ylabel('Average Daily Complaints' if i % n_cols == 0 else '')
    ax.legend(fontsize=9, loc='best')

for j in range(i + 1, len(axes3)):
    axes3[j].set_visible(False)

fig3.suptitle('Louisville Section 1: Normalized Actual Odor Complaints vs Weather',
              fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

Louisville Section 1: Normalized Actual Odor Complaints vs Weather Variables



```
In [11]: # --- SECTION 2: Multivariate Model Marginal Effects Overlay ---
medians_lville = df_count_lville[model_vars].median()

fig4, axes4 = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes4 = axes4.flatten()

for i, var in enumerate(target_weather_vars):
    ax = axes4[i]
    binned_col = f'{var}_binned'
    binned_data = df_count_lville.groupby([binned_col, 'is_weekend']).agg(
        avg_complaints=('complaints', 'mean'),
        num_days=('complaints', 'count')
    ).reset_index()
    binned_data = binned_data[binned_data['num_days'] >= 1]

    # Split weekdays and weekends for background
    wd_data = binned_data[binned_data['is_weekend'] == 0]
    we_data = binned_data[binned_data['is_weekend'] == 1]

    # Background scatter
    ax.scatter(wd_data[binned_col], wd_data['avg_complaints'],
               color='#8b2500', alpha=0.4, edgecolor='k', s=50, label='Weekday actual')
    ax.scatter(we_data[binned_col], we_data['avg_complaints'],
               color='#1f77b4', alpha=0.4, edgecolor='k', s=50, label='Weekend actual')

    # Generate smooth sequence
    var_min = df_count_lville[var].min()
    var_max = df_count_lville[var].max()
    var_seq = np.linspace(var_min, var_max, 200)
```

```

# Weekday Prediction Line
pred_df_wd = pd.DataFrame(index=range(200))
pred_df_wd['const'] = 1.0
for m_var in model_vars:
    if m_var == var:
        pred_df_wd[m_var] = var_seq
    elif var == 'temperature' and m_var == 'temperature_squared':
        pred_df_wd[m_var] = var_seq ** 2
    elif m_var == 'is_weekend':
        pred_df_wd[m_var] = 0
    else:
        pred_df_wd[m_var] = medians_lville[m_var]

pred_X_wd = pred_df_wd[['const'] + model_vars]
pred_y_wd = poisson_res_lville.predict(pred_X_wd)

# Weekend Prediction Line
pred_df_we = pd.DataFrame(index=range(200))
pred_df_we['const'] = 1.0
for m_var in model_vars:
    if m_var == var:
        pred_df_we[m_var] = var_seq
    elif var == 'temperature' and m_var == 'temperature_squared':
        pred_df_we[m_var] = var_seq ** 2
    elif m_var == 'is_weekend':
        pred_df_we[m_var] = 1
    else:
        pred_df_we[m_var] = medians_lville[m_var]

pred_X_we = pred_df_we[['const'] + model_vars]
pred_y_we = poisson_res_lville.predict(pred_X_we)

# Overlay continuous prediction lines
ax.plot(var_seq, pred_y_wd, color='#8b2500', linewidth=2.5, label='Weekd')
ax.plot(var_seq, pred_y_we, color='#1f77b4', linewidth=2.5, label='Weekend')

# Align Y-axis scales
max_actual_wd = wd_data['avg_complaints'].max() if not wd_data.empty else 0
max_actual_we = we_data['avg_complaints'].max() if not we_data.empty else 0
max_y = max(max_actual_wd, max_actual_we, pred_y_wd.max(), pred_y_we.max())
ax.set_ylim(-0.02 * max_y, max_y * 1.08)

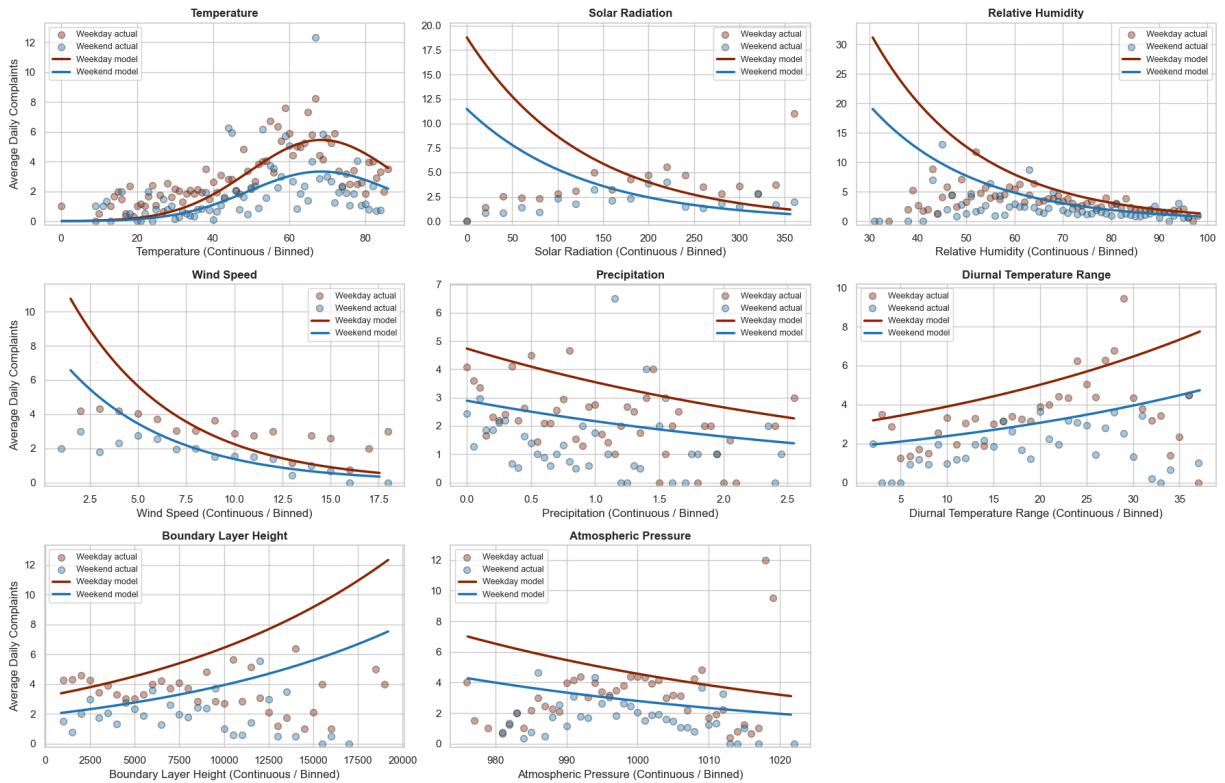
ax.set_title(var.replace('_', ' ').title(), fontsize=12, fontweight='bold')
ax.set_xlabel(var.replace('_', ' ').title() + ' (Continuous / Binned)')
ax.set_ylabel('Average Daily Complaints' if i % n_cols == 0 else '')
ax.legend(fontsize=9, loc='best')

for j in range(i + 1, len(axes4)):
    axes4[j].set_visible(False)

fig4.suptitle('Louisville Section 2: Multivariate Poisson Model Marginal Eff
              ', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```


Louisville Section 2: Multivariate Poisson Model Marginal Effects Overlay



```
In [12]: # --- SECTION 2: Advanced Non-Linear Correlation Metrics ---
print("Calculating advanced correlation metrics for Louisville...")

# 1. Pearson and Spearman correlations
pearson_lvllle = {}
spearman_lvllle = {}
for var in target_weather_vars:
    r_val, _ = pearsonr(df_count_lvllle['complaints'], df_count_lvllle[var])
    rho_val, _ = spearmanr(df_count_lvllle['complaints'], df_count_lvllle[var])
    pearson_lvllle[var] = r_val
    spearman_lvllle[var] = rho_val

# 2. Mutual Information (MI)
X_mi_lvllle = df_count_lvllle[target_weather_vars]
y_mi_lvllle = df_count_lvllle['complaints']
mi_scores_lvllle = mutual_info_regression(X_mi_lvllle, y_mi_lvllle, random_state=42)
mi_lvllle = dict(zip(target_weather_vars, mi_scores_lvllle))

# 3. Model z-scores and Drop-in-Deviance
tvalues_lvllle = poisson_res_lvllle.tvalues
z_lvllle = {}
dev_lvllle = {}
full_dev_lvllle = poisson_res_lvllle.deviance

for var in target_weather_vars:
    if var == 'temperature':
        z_lvllle[var] = max(abs(tvalues_lvllle['temperature']), abs(tvalues_lvllle['temp']))
        vars_keep = [v for v in model_vars if v != 'temperature' and v != 'temp']
    else:
        z_lvllle[var] = abs(tvalues_lvllle[var])
```

```

        vars_keep = [v for v in model_vars if v != var]

        X_red = sm.add_constant(df_count_lville[vars_keep])
        red_model = sm.GLM(y_count_lville, X_red, family=sm.families.Poisson())
        red_res = red_model.fit()
        dev_lville[var] = red_res.deviance - full_dev_lville

    print("Louisville metrics successfully computed!")

```

Calculating advanced correlation metrics for Louisville...
 Louisville metrics successfully computed!

```

In [13]: # --- SECTION 3: Correlation Metrics Visualization ---
lville_comp = pd.DataFrame({
    'Variable': target_weather_vars,
    'Pearson r': [pearson_lville[v] for v in target_weather_vars],
    'Spearman rho': [spearman_lville[v] for v in target_weather_vars],
    'Mutual Information': [mi_lville[v] for v in target_weather_vars],
    'Wald z-score': [z_lville[v] for v in target_weather_vars],
    'Drop-in-Deviance': [dev_lville[v] for v in target_weather_vars]
})

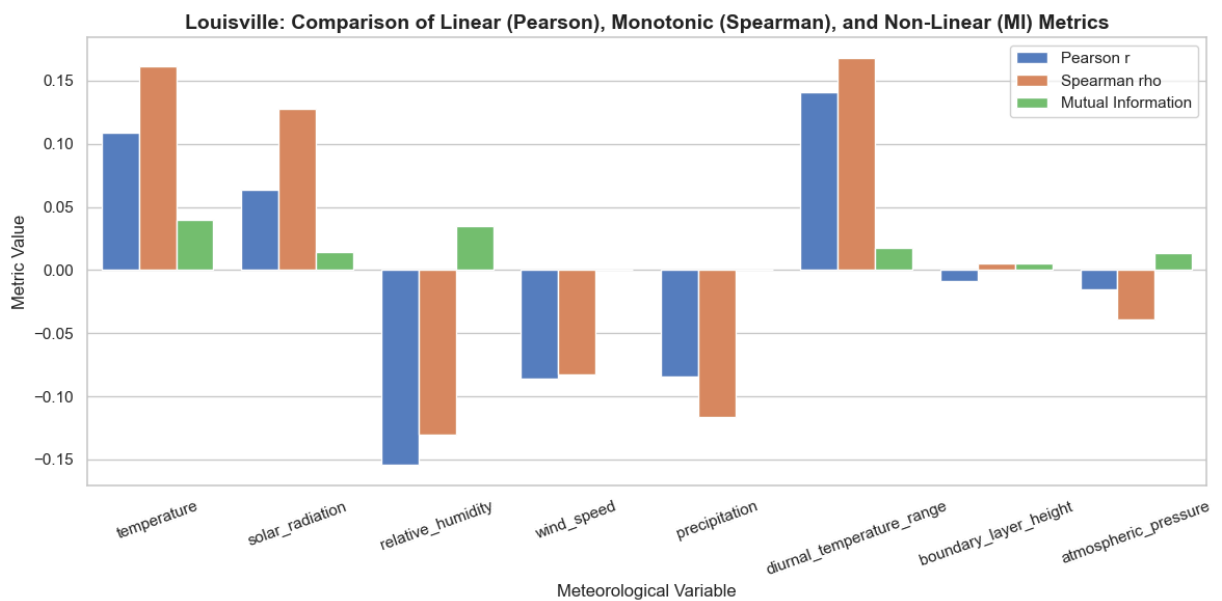
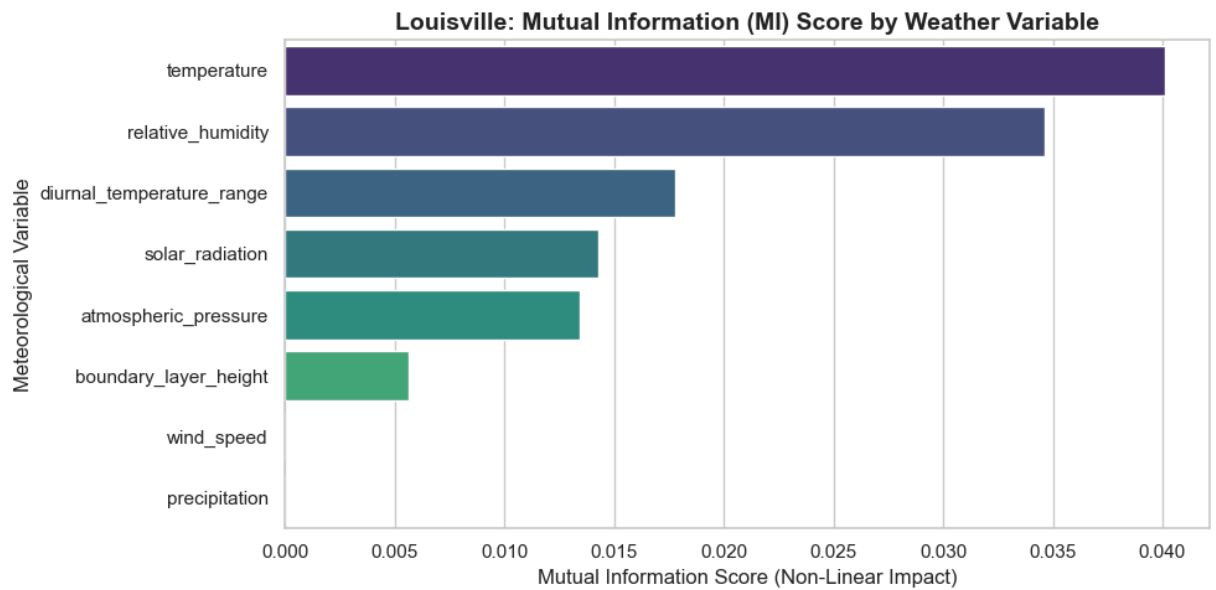
# Horizontal bar chart of Mutual Information (MI)
mi_sorted_l = lville_comp.sort_values(by='Mutual Information', ascending=False)
plt.figure(figsize=(10, 5))
sns.barplot(x='Mutual Information', y='Variable', data=mi_sorted_l, palette=
plt.title('Louisville: Mutual Information (MI) Score by Weather Variable', f
plt.xlabel('Mutual Information Score (Non-Linear Impact)')
plt.ylabel('Meteorological Variable')
plt.tight_layout()
plt.show()

# Melt for comparison of Pearson, Spearman, and MI
melted_lville = lville_comp.melt(id_vars='Variable', value_vars=['Pearson r',
                                                                    'Spearman rho',
                                                                    'Mutual Information',
                                                                    'Wald z-score',
                                                                    'Drop-in-Deviance'],
                                var_name='Metric', value_name='Value')

plt.figure(figsize=(12, 6))
sns.barplot(x='Variable', y='Value', hue='Metric', data=melted_lville, palet
plt.title('Louisville: Comparison of Linear (Pearson), Monotonic (Spearman),
plt.xlabel('Meteorological Variable')
plt.ylabel('Metric Value')
plt.xticks(rotation=20)
plt.legend(loc='best')
plt.tight_layout()
plt.show()

print("Louisville Correlation Metrics Summary Table:")
lville_comp.style.background_gradient(cmap='coolwarm', subset=['Pearson r',
                                                                'Spearman rho',
                                                                'Mutual Information',
                                                                'Wald z-score',
                                                                'Drop-in-Deviance'],
                                .background_gradient(cmap='viridis', subset=['Mutual Informa
                                .format({
                                    'Pearson r': '{:.3f}',
                                    'Spearman rho': '{:.3f}',
                                    'Mutual Information': '{:.3f}',
                                    'Wald z-score': '{:.2f}',
                                    'Drop-in-Deviance': '{:.1f}'
                                })

```



Louisville Correlation Metrics Summary Table:

Out[13]:

	Variable	Pearson r	Spearman rho	Mutual Information	Wald z- score	Drop-in- Deviance
0	temperature	0.108	0.161	0.040	30.01	1821.5
1	solar_radiation	0.064	0.128	0.014	32.01	1046.8
2	relative_humidity	-0.154	-0.131	0.035	34.09	1106.5
3	wind_speed	-0.086	-0.082	0.000	26.47	705.5
4	precipitation	-0.085	-0.116	0.000	5.28	29.8
5	diurnal_temperature_range	0.141	0.168	0.018	10.86	116.0
6	boundary_layer_height	-0.009	0.006	0.006	10.35	105.3
7	atmospheric_pressure	-0.015	-0.039	0.013	7.17	51.3